

Algoritmi e Strutture Dati – 9/1/2024 – Parte B

Esercizio -1 Iscriverti allo scritto entro la scadenza.
In caso di inadempienza, -1 al voto finale.

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

B1 – Commissioni di laurea – Punti ≥ 8

L'ufficio Supporto alla Didattica ha chiesto il vostro aiuto. Il DISI ha realizzato un nuovo regolamento di laurea e comporre le commissioni sta diventando sempre più difficile.

Il problema da risolvere è il seguente:

- esistono k aree didattiche;
- dovete creare k commissioni, una per ogni area didattica;
- esistono n professori; ogni professore i è esperto di un'area didattica p_i ;
- ogni commissione è composta da 5 professori in totale; almeno due di essi devono essere esperti nell'area didattica associata alla commissione; gli altri possono essere esperti oppure no;
- ogni professore può partecipare al massimo a una commissione.

Il vostro compito è trovare un assegnamento fra commissioni e professori che rispetti le regole del DISI. Descrivere un algoritmo che trovi tale assegnamento e discuterne la complessità computazionale.

B2 – Tutti i cicli! – Punti ≥ 10

Scrivere un algoritmo

```
printCycles(GRAPH G, NODE start)
```

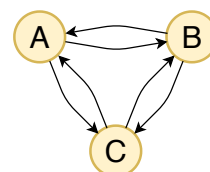
che prenda in input un grafo orientato G e stampi tutti i cicli semplici che contengono $start$.

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale.

In un grafo completo composto da 3 nodi (tutti connessi l'uno con l'altro in entrambe le direzioni), come quello mostrato sotto e dato $start = A$, l'algoritmo deve stampare questi cicli, non necessariamente in quest'ordine:

$$\begin{aligned} A &\rightarrow B \rightarrow A \\ A &\rightarrow C \rightarrow A \\ A &\rightarrow B \rightarrow C \rightarrow A \\ A &\rightarrow C \rightarrow B \rightarrow A \end{aligned}$$

Notate che distinguiamo i cicli $A \rightarrow B \rightarrow C \rightarrow A$ e $A \rightarrow C \rightarrow B \rightarrow A$ che sono formati dagli stessi nodi, ma percorsi in ordine inverso. In altre parole, stiamo considerando cicli orientati.



B3 – Quanti quadrati! – Punti ≥ 12

Scrivere un algoritmo

```
int countSquares(int[][] A, int n)
```

che prenda in input una matrice $n \times n$ contenente valori 0 oppure 1, e restituisca il numero totale di sottomatrici quadrate contenenti solo valori 1.

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale. Esistono soluzioni con complessità $O(n^5)$, $O(n^4)$, $O(n^3)$ e $O(n^2)$.

Per esempio, nella tabella seguente:

0	1	1	1
1	1	1	1
0	1	1	1
0	0	0	0

ci sono 10 sottomatrici quadrate di dimensione 1, 4 sottomatrici quadrate di dimensione 2 e 1 sottomatrice quadrata di dimensione 3, quest'ultima evidenziato in rosso; l'algoritmo deve quindi restituire il valore 15 ($10 + 4 + 1$).